

BCSE204L Design and Analysis of Algorithms

Module-3: Topic: 3: Rabin Karp Algorithm

Dr Sunil Kumar P V

SCOPE

- Assign numeric codes to characters in text and pattern
- Ideally ASCII codes can be used
- However, here we demonstrate using our own codes for simplicity
- Example
 - Text: *aaaaab*
 - Pattern: *aab*
 - Code: $a = 1, b = 2$

- Find a single value for the pattern (called *hash code*) using a *hash function*
- Find hash codes in the text for chunks of pattern length
- If hash codes match, check individual characters, else take next chunk
- Demo → On board

Rolling Hash Function

- Hash codes are to be found out for chunks of pattern length in the main text
- A *rolling hash function* generates the hash code for next chunk from the current chunk in the main text
- Formally, a rolling hash (also known as recursive hashing or rolling checksum) is a hash function where the input is hashed in a window that moves through the input
- Demo → On board, Text: *abcdabce*, Pattern: *bce*

Choosing the right hash function

- The previous 2 examples were simple and took $O(n - m + 1)$ time
- There can be worst case, if such simple hash functions are used
- That can take $O(mn)$ time, equivalent to naive string matching

Choosing the right hash function Example

- Text: *ccaccaaedba*
- Pattern: *dba*
- Demo $\longrightarrow$ On board
- This took $O(mn)$ time because of *spurious hits*
- Better hash functions are to be used to reduce chances of spurious hits

Rabin Karp algorithm

RABIN-KARP-MATCHER(T, P, d, q)

```
1   $n = T.length$ 
2   $m = P.length$ 
3   $h = d^{m-1} \bmod q$ 
4   $p = 0$ 
5   $t_0 = 0$ 
6  for  $i = 1$  to  $m$                                 // preprocessing
7       $p = (dp + P[i]) \bmod q$ 
8       $t_0 = (dt_0 + T[i]) \bmod q$ 
9  for  $s = 0$  to  $n - m$                                 // matching
10     if  $p == t_s$ 
11         if  $P[1..m] == T[s + 1..s + m]$ 
12             print "Pattern occurs with shift"  $s$ 
13     if  $s < n - m$ 
14          $t_{s+1} = (d(t_s - T[s + 1]h) + T[s + m + 1]) \bmod q$ 
```

Usage of Modulo operation

- Hash codes for very long pattern will be huge numbers
- Sometimes beyond system memory limits
- Use a prime number (q) and make all computations in modulo- q arithmetic
- So that the numbers will always be $\leq q$
- But the chances of spurious hits increase by this modification

- $O(n + m - 1)$ in average case, if no spurious hits
- In worst case, all can be spurious hits, despite a carefully defines rolling hash function
- So the worst case complexity is $O(mn)$
- Generally the complexity of Rabin Karp would be between $O(n + m - 1)$ and $O(mn)$

Exercises

- **Exercise-1:** Redo the Rabin Karp example demonstrated in the class with a modulo value, $q = 13$
- **Alert:-** For exam q value will be given the question and you need to use it while solving
- Sometimes numbers will be given directly, rather than codes
- **Exercise-2:** $T = 31415926535$, $P = 26$, $q = 11$, $d = 10$
- **Exercise-3:** Working modulo $q = 13$, how many spurious hits does the Rabin-Karp

Thank you...